

B28: SQL SuperHero Program : Session 7

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

21 February 2025 at 06:31

Virtual Columns

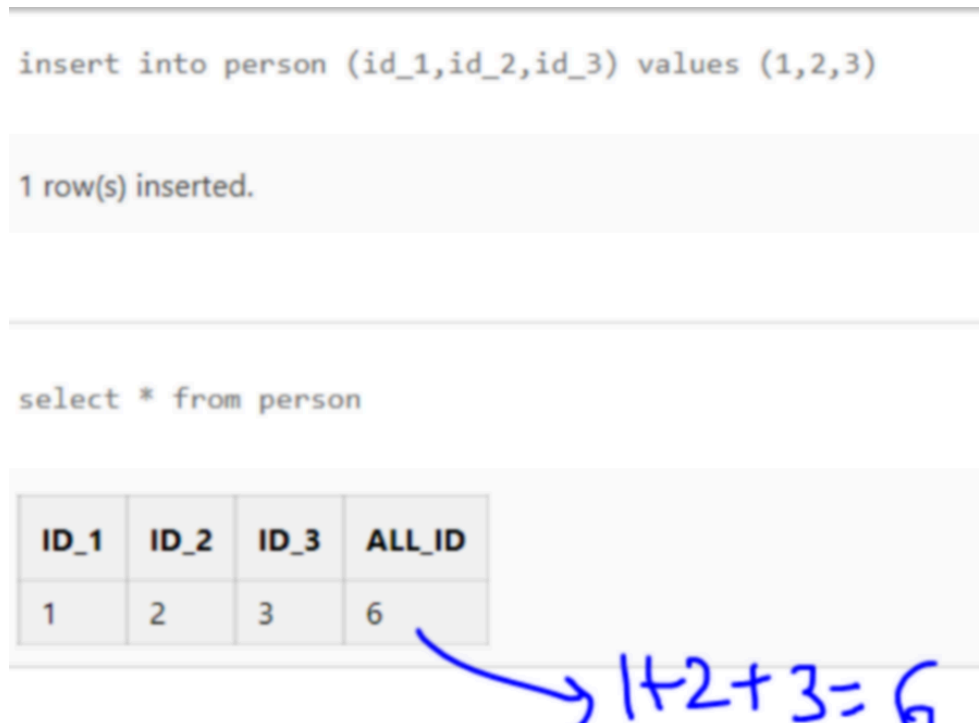
My table has 4 columns. I want to insert data in 3 columns and wants the 4th column will be auto populate by add all these 3

Answer will be: when create the table use the concept of virtual column
That we have to define always generated as (what ever expression)

```
create table PERSON (  
id_1 number, id_2 number, id_3 number,  
all_id GENERATED ALWAYS AS (id_1+id_2+id_3) virtual  
);
```

```
insert into person(id_1,id_2,id_3) values (1,2,3);
```

```
select * from person;
```



The screenshot displays the execution of SQL commands in a database environment. It shows an insert statement for a table named 'person' with columns 'id_1', 'id_2', and 'id_3'. The values (1, 2, 3) are inserted. A message confirms '1 row(s) inserted.' Below this, a query 'select * from person' is executed, resulting in a table with four columns: 'ID_1', 'ID_2', 'ID_3', and 'ALL_ID'. The first row contains the values 1, 2, 3, and 6. A handwritten blue arrow points from the 'ALL_ID' value 6 to the handwritten equation $1+2+3=6$.

ID_1	ID_2	ID_3	ALL_ID
1	2	3	6

Can we add virtual column by alter table command on any existing column

YES

```
CREATE TABLE t1 (col1 number);  
ALTER TABLE t1 ADD col2 number GENERATED ALWAYS AS (col1 + 1);
```

```

1 CREATE TABLE t1 (col1 number);
2 ALTER TABLE t1 ADD col2 number GENERATED ALWAYS AS (col1 + 1) ;
3
4 insert into t1(col1) values (3)
5 select * from t1

```

COL1	COL2
3	4

Download CSV

Handwritten note: Col1 + 1

Can we make Virtual column as Invisible?

```

2 ALTER TABLE t1 ADD col2 number GENERATED ALWAYS AS (col1 + 1) ;
3
4 insert into t1(col1) values (3)
5 select * from t1
6
7 ALTER TABLE t1 MODIFY col2 INVISIBLE;
8
9 select * from t1
10

```

COL1
3

Download CSV

Handwritten note: make the col as invisible

If My Table already have data, so Can I add a column as virtual and populate data in it?

Yes, we can and it will populate the calculated value for old records as well.

```
select * from rakesh
```

ID
1
2
3

3 rows selected.

} - Table has data

→ Added a new column as virtual

```
alter table rakesh add new_column number generated always as (id+1)
```

Table altered.

```
select * from rakesh
```

ID	NEW_COLUMN
1	2
2	3
3	4

3 rows selected.

→ Data populated for previous data as well

IN And BETWEEN

- IN means exact match.
- IN is an extension of equal to operator
- 1000 values we can pass as comma separated
- IN will not compare NULL
- Between means range of values
- where id In (10,20) --> means id = 10 OR id = 20 (exact match)
- Where id between 10 and 20 means range between 10 and 20. 10,11,12,13,14 -- 20

```
select * from hr.employees where employee_id = 100,101
```

↳ This will fail. = only compare
1:1

ORA-00933: SQL command not properly ended

```
select * from hr.employees where employee_id in(100,101)
```

↳ in is an Extension of =
for multiple comparison

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT
100	Steven	King	SKING	515.123.4567	17-JUN-03	AD_PRES	24000	-
101	Neena	Kochhar	NKOCHHAR	515.123.4568	21-SEP-05	AD_VP	17000	-

```
select * from hr.employees where manager_id in (100,102,null)
```

↳ if will not return for
NULL

→ Bcoz Null can not compare

```
select * from hr.employees where department_id = 90 and manager_id is null
```

Null always compare
with IS

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY
100	Steven	King	SKING	515.123.4567	17-JUN-03	AD_PRES	24000

Difference between UNION and UNION ALL

UNION gives us distinct values and UNION ALL gives us duplicate value. UNION Sort the data as well.

Suppose we have 2 tables. test1 and test2. test1 contains id as 1,2,3,4
and test2 contains id as 2,3,4,5

when we do
select * from test1
union
select * from test2

--> it gives us unique records from both data sets. it will give output is 1,2,3,4,5

when we do

```
select * from test1  
UNION ALL  
select * from test2
```

--> **union all**, will not take care of any uniqueness, it will just took data from 1 dataset and combine as it is with another **dataset**

output will be

1,2,3,4,2,3,4,5

```
select * from t11
```

ID
5
9
6
11
9
10

6 rows selected.

```
select * from t12
```

ID
3
7
10

3 rows selected.

```
select * from t11
union
select * from t12
```

→ if you check t11 & t12

ID
3
5
6
7
9
10
11

7 rows selected.

→ Duplicate
Remove d
+ sorted

```
select * from t11
union all
select * from t12
```

ID
5
9
6
11
9
10
3
7
10

9 rows selected.

← As it is.
Just combine both

> Always give examples when someone asks the difference between UNION and UNION ALL.

Number

```
select 1 from dual
union
select '1' from dual
```

Mismatch data type
↓
NO UNION

Varchar

ORA-01790: expression must have same datatype as corresponding expression

More Details: <https://docs.oracle.com/error-help/db/ora-01790>

2 thing in select

```
select '1', 2 from dual
union
select '1' from dual
```

Mismatch
↓
NO UNION

1 thing in select

ORA-01789: query block has incorrect number of result columns

More Details: <https://docs.oracle.com/error-help/db/ora-01789>

=====
--> Always remember, UNION means , same data type and same number of column
=====

```
select '1', 2 from dual
union
select '1', 3 from dual
```

Both have 2 thing in select
& same data type

'1'	2
1	2
1	3

2 rows selected.

→ 3 Column in select

```
select '1', 2, 3 from dual
union
select '1', 3, null from dual
```

Added null to make total 3 column in select

'1'	2	3
1	2	3
1	3	-

2 rows selected.

=====

Added 2 null values in test 2 table:

```
insert into test2 values (null);
insert into test2 values (null);
```

Now if we do **UNION ALL**: Union all, does not care. It will take record from both dataset and just show together

```
select * from test1
union all
select * from test2;
```

ID
1
2
3
4
5
2
3
4
2
-
-

11 rows selected.

Test 1

Test 2

Both null appeared

let me do UNION. : **UNION** remove duplicate and ordered the data and considers null only one time.

```
select * from test1 union select * from test2
```

ID
1
2
3
4
5
-

6 rows selected.

→ Null only 1 time

=====

--> **when we use union and when union all**

if we have a need to present in report or need a dataset from 2 different sources and we have to display only unique records we use UNION, else UNION all if we have to show all data as it is

UNION Vs Distinct :

Distinct will give us unique records and remove duplicates when we are working on one column/One data set. UNION will be the same and remove duplicates but use 2 different data sets.

Select Distinct Id From Table A

UNION

Select Distinct Id From Table B;

The above code is never advisable because distinct is a costly/time consuming operation. If we have to do UNION , so never use it to fetch distinct records, it will do the same task 2 times.

Performance wise UNION VS UNION ALL

- UNION ALL will perform better than UNION when you're not concerned about eliminating duplicate records because you're avoiding an expensive distinct sort operation.
- UNION implements internally 3 queries. 1.SELECT which will return a dataset 2.DISTINCT. And the DISTINCT clause is extremely costly in terms of processing. 3. it sort the data in order